

THREE COLORS OF MADNESS

Technical Design Document

Internal working document. Not for external distribution. Companion to the Design Bible.

Dejunai

Contents

Part One — What This Document Is

Part Two — Current Build

Part Three — Architecture As Built

Part Four — Architecture Decisions Needed

Part Five — Open Production Questions

Part Six — Next Steps

Part One — What This Document Is

This is the Technical Design Document (TDD) for Three Colors of Madness. It is the companion to the Design Bible, not a replacement for any part of it.

The division between the two documents is simple. The bible owns the what and the why — thesis, Design Laws, narrative, the meaning of every mechanic. This document owns the how — engine architecture, data schemas, production sequencing, and the specific open questions a playtest still needs to settle before a mechanism the bible describes gets its final shape. Where a passage in the bible states a mechanism rather than just a rule, that is deliberate: the mechanism itself is what's enforcing a Design Law, and this document does not relitigate it. Where the two documents ever appear to disagree, the bible is correct and this document is wrong until revised — never the reverse.

This document is expected to change often and change fast. Unlike the bible, it has no standing rule against being provisional — an architecture decision made here can be wrong, discovered to be wrong, and replaced without that being a failure of the document. Its job is to stay honest about the current state of the build, not to be permanently settled.

This revision grounds the document in a second, more advanced vertical slice: a continuation of the original Godot 4 prototype, carried forward by the same AI coding agent (Codex) and

brought current against Design Bible v13 and this document's v1. Everything in Parts Two and Three below is a description of what that build actually does, verified by reading its source directly — not a plan for what it should someday do. Where something described in the v1 revision of this document has since changed, been resolved, or been superseded, that is called out explicitly rather than silently overwritten, so the document's own history of decisions stays legible.

Part Two — Current Build

Engine and Target

Godot 4.7, using the GL Compatibility renderer (not Forward+), targeting broad hardware reach over cutting-edge rendering fidelity. Window target 1440×900, canvas-items stretch mode. No import dependencies beyond the engine itself — the project runs from a portable Godot install with no additional libraries or network connection required. Unchanged from the prior slice.

Scope of the Current Slice

The build now covers the opening movement of Chapter One ("No Exit Wound") — the Ophion estate grounds, Pickman Street, the precinct, the boardinghouse, Walter's room — plus a new addition beyond the town sequence: a service-passage encounter beneath the estate's kitchen wing. This is the tunnel referenced in the bible's Observers material, and it is the first part of the build to introduce real player risk rather than pure observation and conversation.

The service-passage encounter is explicitly scoped by its own authors as a bounded interaction test, not the chapter's fatal ending. It works like this: a deterministic 12-second cycle governs a figure standing in the passage — 2 seconds warning (a cough, the head lifting), 4 seconds watching, 6 seconds turned away. Continuous exposure of 1.4 seconds while both watched and in the figure's sightline is fatal. A long outer route and a short right-hand recess are physically safe regardless of Perception; a third, shorter route along reflected wear marks opens only at Perception 4 or higher — the first place in the build where the case-state perception formula (Part Three) gates something other than dialogue text. The flask narrows the passage's shadow presentation and reduces optical strain; it never changes detection or movement, matching the bible's stat-honesty rule (Design Law 11) exactly. Death offers a checkpoint retry, restoring the full record, resources, presentation relief, camera, and encounter clock/exposure to the state captured at the passage's entry.

It is still an 8–12 minute exploratory pacing target for the opening alone — the passage adds a bounded test beyond that, not a chapter-length addition.

Implemented, in addition to everything the prior slice had: the service-passage encounter above, its checkpoint/death/retry loop, a Perception-gated third route, a geometric (non-optical)

strain presentation for the hazard specifically, and a further-refined report/supplement system that now snapshots each claim's *source*, not just its content ([report_sources/county_sources](#) — see Part Three).

Not yet implemented: combat, the forced-spill mechanic, the glass-shattering ontological break, the fatal-comprehension ending, Chapters Two and Three, gamepad support, full key rebinding, a full encumbrance/leveling system, and Ekon's later consumption of Walter's record. The Part Seven difficulty screen is deferred. This is stated plainly in the build's own documentation, not inferred — the passage encounter is deliberately bounded short of the chapter's actual climax.

Design Bible version note: this slice is current against Design Bible v13 and this document's v1, resolving the earlier v10 caveat. Everything the bible added since v10 — the Player Orientation sections, Laws 13–14, the thinning pass that moved production notes into this document — is fair ground to check this build against.

Controls and Accessibility

WASD/arrow movement, mouse look, Q/R camera orbit, scroll-wheel camera distance, Shift for brisk walking, E/F to examine or speak, Tab/I for personal effects, J for the case file, Escape to pause, F11 fullscreen. Menus support mouse or keyboard (Tab, Shift+Tab, Enter/Space). Unchanged from the prior slice.

Accessibility settings (static grain default, distortion intensity, grain intensity, contrast, text size to 130%, mouse sensitivity/inversion, optional interaction markers) are unchanged in substance but now pass through a versioned preference envelope ([scripts/shared/accessibility_settings.gd](#)) that validates ranges, rejects a mismatched schema version outright rather than guessing, and imports the older flat settings format used by the prior slice. This resolves the "settings persistence" item the v1 revision of this document flagged as an open architecture decision (see Part Four). The service passage adds a provisional synthetic cough with a matching readable caption; full audio design remains unfinished elsewhere.

Saves

Godot's user data directory ([.runtime-data](#)) still holds save state, written through a new chapter-neutral module ([scripts/shared/save_store.gd](#)) that performs atomic writes (write to a temp file, then rename) and full-precision JSON round trips — floating-point exactness matters here because encounter timing (the hazard clock and exposure) and remaining presentation relief have to survive a save/load cycle exactly, not approximately.

The reusable case record ([case_state.gd](#)) and the service-passage encounter's checkpoint state are saved separately and deliberately: the case record — evidence, statements, reports, supplements — is chapter-agnostic and reusable across chapters; the encounter's checkpoint

(`tunnel_checkpoint`, `tunnel_dead`, the hazard clock and exposure) is chapter-specific state owned by `scripts/chapters/chapter_one.gd` and written through the same save-store module rather than folded into `case_state.gd`. This is a real, working instance of the reuse boundary Part Four's prior revision asked for: shared infrastructure, chapter-owned state.

The checkpoint itself is captured at the passage's entry. Dying saves the dead state; retrying restores the separate entry snapshot rather than the death state. Mid-encounter saves restore clock, exposure, position, flask relief, camera, and the case record together. Opening any reading pauses world time, including the hazard clock. QA/automated checks use a separate save slot so no automated run can consume a player's live save.

Case-state schema is still `VERSION = 3`, and now explicitly accepts saves at versions 1, 2, or 3, migrating a version-1 save's completed-opening flag into the version-3 `world/estate_complete` shape on load, the same defensive pattern as before, now exercised across one more schema step.

Verification

The build's own automated checks now include `Test interaction loop.cmd` (backed by `tests/interaction_loop.gd`), which verifies both the outer and Perception-gated routes through the passage, empty-flask completion, the county-dispatch consequence chain, historical evidence-source snapshots, the death/continue/retry loop, exact save round trips, paused reading during the encounter, and presentation relief with distortion disabled. `Test town.cmd` and `Test opening.cmd` remain as regression checks against everything the prior slice validated. `tests/walk_driver.gd` and `tests/capture_views.gd` are new supporting harness files for scripted traversal and renderer capture respectively.

Live-renderer QA captures under `docs/qa/` now include `tunnel_access`, `tunnel_death`, and `tunnel_record` alongside the existing set. A new `docs/ARCHITECTURE.md` file, written by the build's own authors, documents the active scene graph, the shared/chapter-specific reuse boundary, the encounter's exact timing rules, and — usefully — states its own limits directly: "The forced spill, glass rupture, combat, full audio, and fatal comprehension sequence remain future work." That kind of built-in honesty about scope is worth preserving as a documentation habit going forward.

Part Three — Architecture As Built

This section describes what the current codebase actually does, file by file, as read directly from the project. Where something differs from the prior slice's Architecture As Built, that's noted explicitly.

Autoloads

Unchanged: `GameState` (`scripts/game_state.gd`) and `LegacyInputs` (`legacy_inputs.gd`) remain registered for backward compatibility with the deprecated 2.5D prototype and are not part of the active data flow. See Part Four for why `GameState`'s link mechanic still matters despite being legacy.

Entry Point and Main Loop — Resolved Decomposition

This is the most significant change since the prior slice. `main.gd` has been reduced from roughly 1,000 lines to 103: it now owns only the player body, the camera rig, and the raw input map, and its `_ready()` does nothing more than wire those up and hand control to a child `ChapterOne` node (`@export var chapter_path: NodePath`). That child node runs `scripts/chapters/chapter_one.gd` (831 lines) — chapter progression, interaction routing, dialogue sequencing, save/load orchestration, and the encounter integration described above. `scripts/chapters/chapter_one_archive.gd` (156 lines) holds the case-file/inventory panel logic that's specific to Walter's record and terminology, kept separate from the reusable interface layer on purpose.

This directly resolves the "main.gd needs decomposition before Chapter Two" item the prior revision of this document flagged as an open architecture decision (see Part Four). The build's own `docs/ARCHITECTURE.md` states the intended reuse boundary plainly: new chapters can supply their own controller and world while reusing the shared modules below; archive/record panels stay chapter-specific "because their terminology and record semantics belong to Walter's case." That is exactly the shape Part Four asked for, and it is worth treating as a real precedent for how Chapter Two's controller and world should be built, not merely aspirational language.

Shared Modules

A new `scripts/shared/` directory holds infrastructure meant to be chapter-agnostic:

- `chapter_interface.gd` (146 lines) — builds and scales the UI: panels, buttons, labels, at both a serif and sans register.
- `dialogue_sequence.gd` (26 lines) — advances a card sequence and invokes its completion callback exactly once, clearing the callback afterward so a sequence can't double-fire completion.
- `save_store.gd` (30 lines) — the chapter-neutral atomic JSON read/write module described in Part Two's Saves section; it does not know or care what a chapter's payload contains.
- `accessibility_settings.gd` (21 lines) — the versioned preference envelope described in Part Two's Controls and Accessibility section.

- `film_presentation.gd` (26 lines) — applies the shader's grain, contrast, animated-grain, iris, and distortion parameters without touching case state; distortion is explicitly scaled by strain and by whether the player is "relieved" (flask), keeping the presentation layer honestly downstream of gameplay state rather than a parallel source of truth.

Case State

`case_state.gd` keeps the same core shape as the prior slice — `evidence`, `statements`, `visited`, `report/copies/report_evidence/report_statements`, `flask`, `coat`, position/yaw, the progression flags, `supplement_filed/county_dispatched` with their own evidence snapshots, `inquiry_topics`, `supplement_history` — with real additions. `report_sources` and `county_sources` are now dictionaries snapshotting *where* each claim in a report or county dispatch came from, not just which evidence IDs were included — a meaningful strengthening of the "witness claims remain distinguished from corroborated findings" rule the prior slice's README already asserted, now backed by an actual source-tracking field rather than by convention alone. `tunnel_complete` and `county_statements` round out the schema to `VERSION = 3`, which — as noted in Part Two — now accepts and migrates versions 1 and 2 as well.

`perception()` is unchanged: `2 + min(4, evidence.size() / 2)`, still a flat function of evidence count with no linking mechanic. It is no longer a purely cosmetic number, though — the service passage's third route is gated on `perception() >= 4`, so this formula (and the still-open corkboard/link-mechanic question in Part Four) now has a real gameplay consequence beyond dialogue flavor.

The hazard encounter's own runtime state — checkpoint snapshot, death flag, clock, exposure — deliberately does **not** live in `case_state.gd`. It's held and saved by `chapter_one.gd` itself, through the shared save-store module. That's a clean, legible instance of the shared/chapter-specific boundary described above: the case record is what should carry forward into later chapters and Ekon's eventual consumption of it; the encounter clock is not.

Dialogue and Facts

`story.gd` and `town_story.gd` are unchanged — static data tables, still explicit that "film effects never modify these canonical strings," still the code-level implementation of Design Law 4. `tunnel_story.gd` is new but much thinner (14 lines): it holds the passage's factual `FACTS` table (the municipal foundation sheet, the county's follow-up request) and its three-card `ENTRY` sequence, but not the encounter's own moment-to-moment cue text.

That cue text — "It sees Walter. Get behind stone," the phase-dependent warning/watching/turned lines — is generated directly inside `tunnel.gd`'s `cue()` method rather than pulled from a data table the way `story.gd` and `town_story.gd`'s content is. This is a minor but real inconsistency worth naming: everywhere else, canonical text lives in a data table separate from the code that presents it; here, the hazard's live narration is inline in the class that also drives its game logic. It doesn't currently violate Design Law 4 in practice — the text is still purely factual, no presentation bleeding into content — but it's a different pattern from the rest of the codebase, and worth deciding on purpose (extract to a table, or accept that a real-time reactive cue is a legitimate exception to the static-table convention) before a second encounter of this kind gets built.

Environment

`estate.gd` and `town.gd` are unchanged — procedural geometry from primitive shapes, no `.tscn` scene files or imported assets, the same blackout strategy flagged as an open pipeline decision in Part Four (still open — see below). `tunnel.gd` (194 lines) extends `estate.gd` directly, reusing its collision and geometry helpers (`box`, `person`, `target`, `lettering`) rather than duplicating them — a sensible instance of code reuse within the procedural-geometry approach, whichever way that broader pipeline question eventually gets resolved.

Presentation Shader — Changed Scope

This is a regression worth flagging plainly rather than glossing over. The prior slice's active shader (`shaders/film_1923.gdshader`) implemented a working selective-color-preservation device: grayscale by luminance, except where red-channel dominance was detected, blended back in through a `preserve_red` uniform — the actual code-level version of the bible's "grayscale except for color that isn't supposed to be there" Observer device. In this slice, that file has been demoted to legacy status (`docs/ARCHITECTURE.md`: "shaders/film_1923.gdshader belongs to the retained legacy prototype"), and the new active shader (`film.gdshader`, at the project root, 26 lines) is purely monochrome — luminance, contrast, grain, vignette, iris mask, horizontal-wobble distortion, no color channel of any kind survives the fragment shader. The Observer color-tell mechanism, as a working piece of code, currently does not exist in the active build. This needs to come back before the Observers material in Chapter One (the estate grounds crew scene added to the bible this revision) can be staged the way the bible now requires it to be — see Part Four.

On the accessibility side, this slice does add something the prior one didn't have: `tunnel.gd`'s `presentation(strain, relieved)` method scales wall-shadow and strain-shadow geometry directly by strain and by whether the flask has relieved it, independent of the shader's optical distortion entirely. This is a real, working first instance of the "geometric, non-optical

degradation vocabulary" that Part Five's Accessible-Mode Degradation Vocabulary question asks for — see that entry below for what it does and doesn't settle.

Settings

Superseded by `scripts/shared/accessibility_settings.gd`, described above. The flat runtime-only dictionary the prior slice used is now the *import* format that module accepts from an older save, not the live representation.

Part Four — Architecture Decisions Needed

These are not open production questions in the bible's sense (Part Five below covers those) — they are decisions this codebase is already implicitly making by default, and that default should be confirmed or overridden on purpose before Chapter Two's build starts. Items resolved since the prior revision of this document are marked as such rather than deleted, so the decision history stays visible.

Two coexisting case-tracking systems — still open. The legacy `GameState` autoload still implements a link-based corkboard mechanic that the active build doesn't use; `case_state.gd`'s `perception()` is still a flat evidence-count function. This is now less abstract than it was: the service passage's third route gates directly on `perception() >= 4`, so whichever way this question resolves will change what a Perception-4 player can actually reach, not just what dialogue they hear. This should still be decided explicitly — migrate the link mechanic forward, rebuild it fresh, or commit to the flat formula and update the bible's corkboard language to match. It also still directly affects the Minimal-Engagement Staging question in Part Five.

main.gd decomposition — resolved. The prior revision of this document asked for player control, camera, UI, dialogue, save/load, and test-harness integration to be split out of a single ~1,000-line file before Chapter Two's build begins. That has happened: see Part Three's Entry Point and Shared Modules sections. The resulting shape — a thin root scene owning only player/camera, a chapter-owned orchestration node, and chapter-agnostic shared modules — is a reasonable template to hold Chapter Two's controller and world to, rather than a decision to relitigate from scratch.

Procedural geometry: blockout strategy or production pipeline? — still open. Unchanged from the prior revision: `estate.gd/town.gd/tunnel.gd` still generate all geometry from code at runtime, with no authored or imported assets. This still needs to be named explicitly as temporary placeholder or intentional pipeline before Chapter Two's roughly 200 bespoke artifacts get built one way or the other.

Shader color-preservation scope — reopened, more urgent than before. The prior revision flagged this as a scoping question (should red-dominance preservation apply globally or only to flagged Observer objects?). It is now a more basic problem: the currently active shader doesn't preserve any color at all, having dropped the mechanism entirely in favor of the simpler `film.gdshader`. This needs to be restored — informed by the original scoping question, ideally scoped to specific flagged materials rather than reintroduced as a blanket red-dominance filter — before the estate grounds crew's Observer beat (added to the bible this revision) can be built to spec. This is now a build gap, not just an open design question.

Settings persistence — resolved. `scripts/shared/accessibility_settings.gd`'s versioned envelope, with explicit legacy-format import, is exactly the persistence/versioning story the prior revision asked for.

Hazard cue text lives in code, not a data table — new. Flagged in Part Three's Dialogue and Facts section: `tunnel.gd`'s `cue()` method generates the encounter's live narration inline rather than through a `story.gd`-style data table. Worth a deliberate decision — extract it to a table for consistency with the rest of the codebase's Design Law 4 pattern, or treat a real-time reactive hazard cue as a legitimate, documented exception — before a second timed encounter gets built on the same pattern.

Part Five — Open Production Questions

Each remains an open question pending playtest, not a settled design decision — the bible states the constraint each answer must satisfy; this document owns tracking the answer. Updated where this slice adds real evidence.

Chapter One — Minimal-Engagement Staging (Design Bible, Part Three, Mechanics).

Unchanged in substance. Still entangled with the corkboard/link-mechanic decision in Part Four — now somewhat more concretely, since Perception 4 (itself downstream of that decision) already gates one piece of content (the service passage's third route), which is a small working example of "what a diligent player has already pieced together" actually changing what's reachable, not just what's said.

Chapter Two — Procedural Trinket Generation (Design Bible, Part Four, Mechanics).

Unchanged. Not yet prototyped; nothing in this slice bears on it.

Chapter Two — Acquisition/Documentation Mechanical Distinction (Design Bible, Part Four, Player Orientation). Unchanged. Not yet prototyped; nothing in this slice bears on it directly, though the service passage's own risk/reward shape (a longer safe route vs. a shorter, Perception-gated one) is a useful nearby reference for how "two ways to get the same outcome, at different costs" can be built cleanly.

Chapter Three — Match Consumption Model (Design Bible, Part Five, Mechanics).

Unchanged. No Chapter Three slice exists yet to prototype against.

Cross-Chapter — Accessible-Mode Degradation Vocabulary (Design Bible, Part Six, Accessibility Suite) — first data point.

The prior revision noted this had no fallback vocabulary implemented anywhere. It now has one, in miniature: the service passage's `presentation(strain, relieved)` method stretches wall-shadow and strain-shadow geometry in direct proportion to strain, independent of the shader's optical distortion — readable, per the code's own comment, "even at zero optical distortion." This is exactly the shape the bible's requirement calls for (composition and geometry doing the degradation's job instead of optical assault), but it's a single, narrow instance — one hazard, one kind of geometric distortion (shadow scale) — not yet a general vocabulary that could cover Chapter One's other film-grammar moments or Chapters Two and Three. Worth treating as a validated proof of concept to generalize from, not yet the answer to the full question.

Meta Layer — Difficulty-Menu Glitch Cue (Design Bible, Part Seven, Difficulty Selection).

Unchanged. Still not built; no difficulty screen exists in the current slice, explicitly deferred per the build's own README.

Part Six — Next Steps

In rough priority order: restore the Observer color-preservation device in the active shader — this is now a concrete build gap against the bible's Part Two material, not a scoping question to think about later. Resolve the corkboard/link-mechanic decision, which now has a real gameplay consequence via the service passage's Perception gate, not just a dialogue-flavor one. Decide the procedural-art-pipeline question before committing more chapters to code-generated geometry. Decide whether the hazard's inline cue text should move to a data table before a second encounter of this kind is built on the same pattern. Once a Chapter Two slice exists, this document gets a matching Architecture As Built entry for it, and Chapter Two's controller/world split should be held to the shared/chapter-specific template this slice established, not designed from scratch.